

# Relatório de Protótipo para Monitoramento de Vibração em Máquinas Rotativas

Felipe Augusto Ramos Diniz

9 de outubro de 2025

## 1 Introdução

O monitoramento de vibrações é uma técnica consolidada na manutenção preditiva de equipamentos industriais, como conjuntos moto-bomba. A análise dos sinais vibratórios permite detectar alterações no comportamento dinâmico de uma máquina, possibilitando a identificação de falhas, como desbalanceamento, desalinhamento e desgaste em rolamentos. Conforme demonstrado por diversos autores, a análise do espectro de frequência via Transformada Rápida de Fourier (FFT) é uma metodologia eficaz para decompor os sinais complexos de vibração e identificar suas frequências características.

Tradicionalmente, sistemas de monitoramento industrial envolvem alto custo de instrumentação. Este projeto, no entanto, explora a viabilidade de um protótipo de baixo custo, utilizando um microcontrolador ESP32 e um acelerômetro ADXL345. A abordagem é similar à de outros estudos que validaram o uso de plataformas como Arduino e sensores para o diagnóstico de falhas de baixa frequência em equipamentos reais.

Este relatório detalha a metodologia de desenvolvimento e validação do protótipo, utilizando um celular como fonte de vibração controlada para simular a assinatura espectral de uma máquina rotativa.

## 2 Componentes e Justificativas

- **Microcontrolador ESP32 WROOM-32:** Escolhido por seu baixo custo e disponibilidade no laboratório.
- **Acelerômetro ADXL345:** Um sensor MEMS triaxial de baixo consumo, com saída digital e interface de comunicação I2C/SPI. Sua taxa de amostragem configurável (até 3200 Hz) e processamento de sinal embarcado o tornam adequado para a captura de sinais vibratórios dinâmicos.
- **Celular (iPhone 11):** Utilizado como gerador de sinais de referência, produzindo vibrações em frequências específicas por meio de um aplicativo gerador de tons.
- **Software:** Arduino IDE para o firmware do ESP32, Python 3 com as bibliotecas Pandas, NumPy e Matplotlib para coleta e análise de dados no computador.

## 3 Metodologia

A validação do protótipo foi realizada por meio de testes com frequências controladas, geradas pelo aplicativo *Frequency: Sound Generator*. A metodologia foi dividida em

setup experimental, aquisição e processamento dos sinais.

### 3.1 Setup Experimental

O acelerômetro foi acoplado fisicamente à carcaça do celular. Testes iniciais revelaram que o posicionamento do sensor é crítico para a qualidade do sinal, um fato também observado em manuais de boas práticas. A melhor resposta foi obtida quando o sensor foi fixado firmemente com fita dupla-face sobre a principal fonte de vibração do celular (o alto-falante). A montagem final pode ser vista na Figura 1.

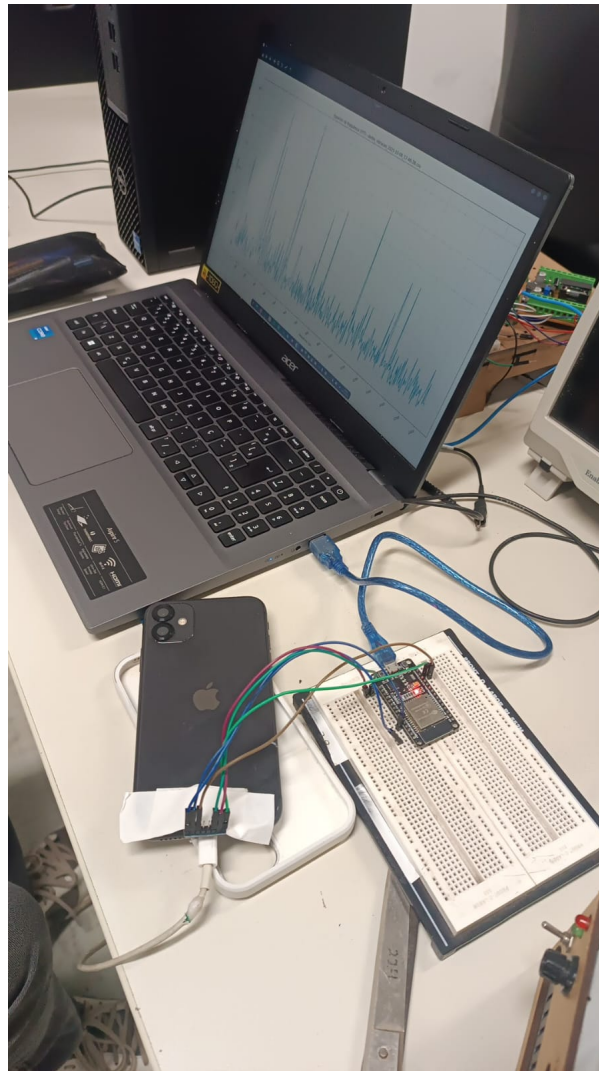


Figura 1: Montagem do sensor ADXL345 acoplado ao celular para o teste.

### 3.2 Aquisição e Processamento de Sinais

O ESP32 foi configurado para operar como um coletor de dados de alta velocidade. Para evitar problemas de sincronização intermitentes, foi implementado um protocolo de "handshake", onde o script em Python aguarda uma mensagem de prontidão do ESP32 antes de enviar um comando para iniciar a coleta.

O sensor foi configurado para uma taxa de amostragem de 3200 Hz, respeitando o Teorema de Nyquist-Shannon para analisar frequências de até 1600 Hz, faixa que abrange os principais fenômenos vibratórios em muitos motores industriais. O processo segue os seguintes passos:

1. Ao receber um comando via serial, o ESP32 coleta um bloco de 1024 amostras dos 3 eixos do acelerômetro, armazenando-as em um buffer interno.
2. O bloco de dados é transmitido em formato binário para um script em Python.
3. O script desempacota os bytes, converte os valores para aceleração ( $m/s^2$ ) e os armazena em um arquivo CSV com timestamps de alta resolução.
4. Um segundo script Python carrega o arquivo CSV e aplica a Transformada Rápida de Fourier (FFT) para gerar o espectro de frequência do sinal.

## 4 Resultados e Discussão

A validação do protótipo, apesar de sua natureza rudimentar, confirmou a eficácia da metodologia de aquisição e análise de vibrações. Um dos aprendizados mais significativos foi a importância crítica do posicionamento e acoplamento do sensor, um ponto enfatizado em manuais de boas práticas de medição de vibração. A diferença entre um acoplamento mecânico inadequado e um otimizado é demonstrada visualmente nos testes a seguir.

### 4.1 Análise do Sinal de 30 Hz

No primeiro teste, foi gerado um sinal de 30 Hz. A Figura 4 compara o espectro de frequência obtido com o sensor posicionado no centro da carcaça do celular versus posicionado diretamente sobre a fonte sonora.

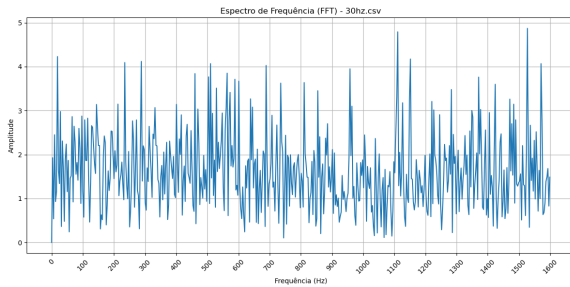


Figura 2: \*  
(a) Espectro inicial (ruidoso)

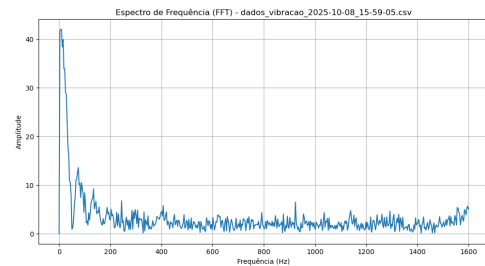


Figura 3: \*  
(b) Espectro otimizado

Figura 4: Comparação do espectro para um sinal de 30 Hz. Em (a), o sinal com alto ruído obtido com o sensor no centro da carcaça. Em (b), o espectro claro com um pico fundamental dominante em 30 Hz, obtido após o reposicionamento do sensor.

Como se pode observar, o espectro inicial (a) é dominado por ruído, tornando a identificação da frequência de 30 Hz ambígua. Após otimizar o posicionamento para garantir um melhor acoplamento mecânico, o espectro resultante (b) mostra um pico nítido e de alta amplitude exatamente na frequência de interesse, validando a capacidade de detecção do sistema.

### 4.2 Análise do Sinal de 440 Hz

O mesmo procedimento foi repetido para um sinal de 440 Hz (Figura 7), que revelou um cenário ainda mais rico.

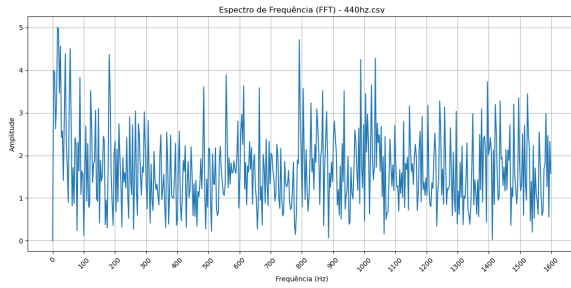


Figura 5: \*  
(a) Espectro inicial (ruidoso)

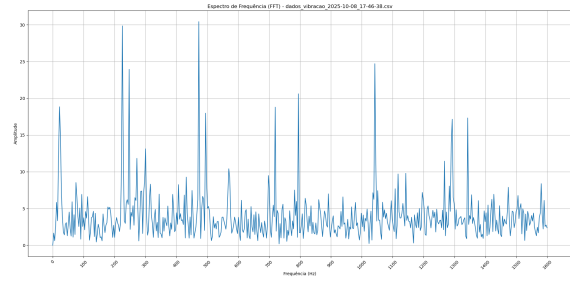


Figura 6: \*  
(b) Espectro otimizado

Figura 7: Comparação do espectro para um sinal de 440 Hz. Em (a), o sinal ruidoso onde a frequência fundamental é indistinguível. Em (b), a clara identificação do pico em 440 Hz e de outros fenômenos secundários.

Novamente, o resultado inicial (a) foi inconclusivo. No entanto, o espectro otimizado (b) não apenas identificou o pico fundamental em **440 Hz**, mas também outros fenômenos complexos. Foram claramente identificados picos nos múltiplos inteiros de **880 Hz** ( $2f$ ) e **1320 Hz** ( $3f$ ), que são os **harmônicos** da frequência fundamental. A análise de harmônicos é uma ferramenta padrão para diagnóstico de desalinhamento e folgas mecânicas em máquinas reais. Adicionalmente, o pico em  $\approx 220$  Hz ( $f/2$ ) é característico de uma **resposta não-linear** do sistema mecânico, demonstrando a sensibilidade do protótipo a comportamentos dinâmicos complexos.

Talvez a descoberta mais relevante tenha sido a observação do fenômeno da **ressonância**, onde a vibração de entrada excita frequências naturais da estrutura em pontos de maior energia. Este fenômeno é precisamente o princípio chave utilizado para detectar falhas em rolamentos em estágios iniciais, conforme descrito na literatura. Os impactos de um rolamento defeituoso (evento de baixa frequência) são fracos, mas a ressonância estrutural que eles excitam em alta frequência é um indicador claro e de alta energia, facilmente detectável por uma análise de FFT.

As principais limitações do protótipo residem na sua montagem improvisada. No entanto, como ferramenta de validação, o sistema demonstrou com sucesso que a combinação do ESP32 com o ADXL345 é adequada para capturar e processar sinais vibratórios complexos, fornecendo uma base sólida para a aplicação da metodologia em motores elétricos reais.

## 5 Conclusão

O protótipo desenvolvido demonstrou ser uma ferramenta viável e de baixo custo para a aquisição e análise espectral de sinais vibratórios. Os testes confirmaram a capacidade do sistema em identificar com precisão frequências fundamentais, harmônicos e ressonâncias, validando a metodologia para futuras aplicações em equipamentos reais. O projeto corrobora as conclusões de outros estudos sobre a eficácia de sistemas baseados em microcontroladores e sensores MEMS para tarefas de manutenção preditiva.

O próximo passo é a aplicação desta ferramenta validada no conjunto motor-bomba real para capturar sua assinatura espectral em operação normal e monitorar a evolução de seus picos de frequência ao longo do tempo.

## Referências

- [1] F. J. da Silva, *Manutenção Preditiva por Análise de Vibração com Utilização de Micro-controlador Arduino e Sensor MEMS: Estudo de Caso de um Conjunto Moto-Bomba*. Trabalho de Conclusão de Curso, Universidade Federal da Grande Dourados, 2022.
- [2] R. F. M. Marçal e A. A. Susin, "Detectando Falhas Incipientes Em Máquinas Rotativas," *Revista Gestão Industrial*, vol. 01, n. 02, pp. 83–92, 2005.
- [3] V. C. P. da Silva, *O Emprego do Acelerômetro ADXL345 para a Obtenção da Aceleração, Velocidade e Deslocamento no Eixo XY*. Trabalho de Conclusão de Curso, Universidade de São Paulo, 2018.

## A Anexo A: Código Arduino IDE (Coleta de Dados no ESP32)

```
1 #include <Wire.h>
2 #include <Adafruit_Sensor.h>
3 #include <Adafruit_ADXL345_U.h>
4 #include "rtc_wdt.h"
5 Adafruit_ADXL345_Unified accel = Adafruit_ADXL345_Unified();
6 #define NUM_AMOSTRAS 1024
7 struct AmostraAcelerometro {
8     int16_t x;
9     int16_t y;
10    int16_t z;
11 };
12 AmostraAcelerometro bufferDeAmostras[NUM_AMOSTRAS];
13
14 void setup() {
15     Serial.begin(115200);
16     delay(2000); // Pausa para estabiliza o
17
18     if (!accel.begin()) {
19         Serial.println("ADXL345 nao detectado!");
20         while (1); // Trava aqui se o sensor n o for encontrado
21     }
22     accel.setDataRate(ADXL345_DATARATE_3200_HZ);
23     accel.setRange(ADXL345_RANGE_16_G);
24
25     Serial.println("Iniciando coleta cont nua...");
26 }
27
28 void loop() {
29     // --- Coleta ---
30     for (int i = 0; i < NUM_AMOSTRAS; i++) {
31         sensors_event_t event;
32         accel.getEvent(&event);
33         bufferDeAmostras[i].x = event.acceleration.x * 100;
34         bufferDeAmostras[i].y = event.acceleration.y * 100;
35         bufferDeAmostras[i].z = event.acceleration.z * 100;
36         if (i % 100 == 0) rtc_wdt_feed();
```

```

37 }
38
39 // --- Envio ---
40 for (int i = 0; i < NUM_AMOSTRAS; i++) {
41     while (Serial.availableForWrite() < sizeof(AmostraAcelerometro)
42         ) {
43         delay(1);
44     }
45     Serial.write((uint8_t*)&bufferDeAmostras[i], sizeof(
46         AmostraAcelerometro));
47 }
48
49 delay(1000); // Pausa de 1 segundo antes de coletar o pr ximo
50 bloco
51 }

```

## B Anexo B: Código Python para Recebimento e Armazenamento

```

1 import serial
2 import struct
3 import time
4 import csv
5 from datetime import datetime, timedelta
6
7 # --- Configura es ---
8 PORTA_SERIAL = '/dev/ttyACM0'
9 VELOCIDADE_SERIAL = 115200
10 NUM_AMOSTRAS = 1024
11 TAXA_SENSOR_HZ = 3200.0 # Taxa de amostragem configurada no ESP32
12
13 # Cria um nome de arquivo nico com a data e hora atuais
14 nome_arquivo = f"dados_vibracao_{datetime.now().strftime('%Y-%m-%d_%H-%M-%S')}.csv"
15
16 print("--- INICIANDO LOGGER DE DADOS ---")
17 try:
18     # Abre a porta serial com timeout longo e sem reset DTR
19     ser = serial.Serial(PORTA_SERIAL, VELOCIDADE_SERIAL, timeout
20         =20, dsrdtr=False)
21     print(f"Porta {PORTA_SERIAL} aberta. Aguardando o ESP32
22         estabilizar (3 segundos)...")
23     time.sleep(3)
24     ser.reset_input_buffer() # Limpa qualquer dado antigo
25
26     print("Aguardando o pr ximo bloco de dados do ESP32...")
27     bytes_esperados = NUM_AMOSTRAS * 3 * 2 # 1024 amostras * 3
28         eixos * 2 bytes/eixo
29     dados_binarios = ser.read(bytes_esperados)

```

```

28     print(f"Leitura finalizada. Recebidos {len(dados_binarios)} de
        {bytes_esperados} bytes.")
29
30     # Se a leitura foi bem-sucedida, processa e salva os dados
31     if len(dados_binarios) == bytes_esperados:
32         print("SUCESSO! Processando e salvando os dados no CSV...")
33
34         # Desempacota o bloco de bytes para uma lista de n meros
35         formato_struct = '<' + ('hhh' * NUM_AMOSTRAS) # '<' para
            little-endian
36         valores_desempacotados = struct.unpack(formato_struct,
            dados_binarios)
37
38         # Calcula o pequeno intervalo de tempo entre cada amostra
39         delta_por_amostra = timedelta(seconds=1/TAXA_SENSOR_HZ)
40         timestamp_atual = datetime.now()
41
42         # Abre o arquivo CSV para escrita
43         with open(nome_arquivo, 'w', newline='') as f:
44             writer = csv.writer(f)
45             writer.writerow(['timestamp', 'x_accel', 'y_accel', '
                z_accel'])
46
47             # Escreve cada amostra no arquivo com um timestamp
                preciso
48             for i in range(NUM_AMOSTRAS):
49                 idx = i * 3
50                 x = valores_desempacotados[idx] / 100.0
51                 y = valores_desempacotados[idx+1] / 100.0
52                 z = valores_desempacotados[idx+2] / 100.0
53
54                 timestamp_linha = timestamp_atual + (i *
                    delta_por_amostra)
55                 writer.writerow([timestamp_linha.isoformat(), x, y,
                    z])
56
57             print(f"Dados salvos com sucesso em {nome_arquivo}")
58         else:
59             print("FALHA. O bloco de dados n o foi recebido. Verifique
                o cabo e as conex es f sicas.")
60
61     except Exception as e:
62         print(f"Ocorreu um erro: {e}")
63     finally:
64         if 'ser' in locals() and ser.is_open:
65             ser.close()
66             print("Conex o serial fechada.")

```

## C Anexo C: Código Python para Análise FFT

```

1 import pandas as pd
2 import numpy as np

```

```

3 import matplotlib.pyplot as plt
4 import glob
5 import os
6
7 def analisar_vibracao(nome_arquivo_csv):
8     """
9     Carrega dados de vibra o , calcula a FFT, plota o espectro e
10     imprime a frequ ncia fundamental.
11     """
12     try:
13         print(f"\nCarregando e analisando '{nome_arquivo_csv}'...")
14         df = pd.read_csv(nome_arquivo_csv)
15         sinal = df['z_accel'].values
16
17         df['timestamp'] = pd.to_datetime(df['timestamp'])
18         intervalo_tempo = df['timestamp'].diff().mean().
19             total_seconds()
20         taxa_amostragem = 1 / intervalo_tempo
21         print(f"Taxa de amostragem calculada: {taxa_amostragem:.2f}
22             Hz")
23
24         N = len(sinal)
25         sinal_normalizado = sinal - np.mean(sinal)
26
27         yf = np.fft.fft(sinal_normalizado)
28         xf = np.fft.fftfreq(N, 1 / taxa_amostragem)
29
30         # --- NOVO: Encontra e imprime a frequ ncia fundamental
31         ---
32
33         # Pegamos as amplitudes e frequ ncias correspondentes (
34         apenas a parte positiva)
35         # Ignoramos o ndice 0, que o componente DC (
36         frequ ncia zero)
37         amplitudes = 2.0/N * np.abs(yf[1:N//2])
38         frequencias = xf[1:N//2]
39
40         # Encontra o ndice da maior amplitude
41         indice_pico = np.argmax(amplitudes)
42
43         # Pega a frequ ncia e a amplitude correspondentes a esse
44         ndice
45         frequencia_fundamental = frequencias[indice_pico]
46         amplitude_pico = amplitudes[indice_pico]
47
48         print("\n--- An lise de Frequ ncia ---")
49         print(f"Frequ ncia Fundamental (Pico Principal): {
50             frequencia_fundamental:.2f} Hz")
51         print(f"Amplitude do Pico: {amplitude_pico:.2f}")
52         print("-----")
53
54         # --- Prepara o e Exibi o do Gr fico ---
55         plt.figure(figsize=(12, 6))

```



```

48     plt.plot(frequencias, amplitudes) # Usando as variáveis
      que j criamos
49
50     plt.title(f'Espectro de Frequência (FFT) - {os.path.
      basename(nome_arquivo_csv)}')
51     plt.xlabel('Frequência (Hz)')
52     plt.ylabel('Amplitude')
53     plt.grid(True)
54
55     # Marca o pico fundamental no gráfico para fácil
      identificação
56     plt.plot(frequencia_fundamental, amplitude_pico, "x",
      markersize=10, label=f"Pico em {frequencia_fundamental
      :.2f} Hz")
57     plt.legend()
58
59     max_frequencia = taxa_amostragem / 2
60     ticks = np.arange(0, max_frequencia + 1, 100)
61     plt.xticks(ticks, rotation=45)
62
63     plt.tight_layout()
64     plt.show()
65
66 except Exception as e:
67     print(f"Ocorreu um erro durante a análise: {e}")
68
69
70
71 if __name__ == "__main__":
72
73     arquivos_csv = glob.glob('*.csv')
74
75     if not arquivos_csv:
76         print("Nenhum arquivo .csv encontrado nesta pasta.")
77     else:
78         print("Arquivos CSV encontrados:")
79         for i, nome_arquivo in enumerate(arquivos_csv):
80             print(f" [{i+1}] {nome_arquivo}")
81
82         try:
83
84             escolha = int(input("\nDigite o número do arquivo que
      deseja analisar: "))
85
86             if 1 <= escolha <= len(arquivos_csv):
87                 arquivo_selecionado = arquivos_csv[escolha - 1]
88                 analisar_vibração(arquivo_selecionado)
89             else:
90                 print("Escolha inválida.")
91         except ValueError:
92             print("Entrada inválida. Por favor, digite um número.
      ")

```